DevOps Docker Lab: Code & Architecture Reference

Visual reference guide containing folder trees, Dockerfiles, Compose files, and proxy configurations.

1. Repository Structure & Artifact Tree

One-line explanation: A clean, modular multi-tier Docker curriculum spanning single-stage to hardened proxy stacks.

```
docker-flask-react-lab/
  |-- 01-flask-basic/        # Single-stage Flask container (python:3.12-slim)
  |-- 02-flask-multistage/   # Multi-stage Flask with venv, non-root user, Gunicorn
  |-- 03-three-tier/         # Compose stack: React + Flask + PostgreSQL with named volume
  |-- 04-three-tier-nginx/   # Production stack: Nginx proxy + isolated dual networks
  \-- docs/                  # Runbook (step-by-step.md) and PDF documentation
```

2. 01-flask-basic/Dockerfile

One-line explanation: A single-stage Dockerfile packaging a Python 3.12 Flask API into an official python:3.12-slim base.

```
FROM python:3.12-slim

# Set container working directory
WORKDIR /app

# Copy dependencies file first for Docker layer caching
COPY requirements.txt .

# Install Python dependencies without caching wheels
RUN pip install --no-cache-dir -r requirements.txt

# Copy application source code
COPY app.py .

# Expose Flask port 5000
EXPOSE 5000
CMD ["python", "app.py"]
```

3. 02-flask-multistage/Dockerfile

One-line explanation: Multi-stage Dockerfile isolating compilation into a virtual environment and running as non-root with Gunicorn.

```
# Stage 1: Build stage (compile & install dependencies)
FROM python:3.12-slim AS builder
WORKDIR /build
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Stage 2: Runtime stage (clean & lightweight)
FROM python:3.12-slim AS runner
RUN useradd --create-home --shell /bin/bash appuser
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY app.py .
RUN chown -R appuser:appuser /app
USER appuser
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

4. 03-three-tier/backend/Dockerfile

One-line explanation: Multi-stage backend Dockerfile installing psycopg2-binary and running Gunicorn as non-root user.

```
# Multi-stage build mirroring 02-flask-multistage with psycopg2-binary
FROM python:3.12-slim AS builder
WORKDIR /build
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim AS runner
RUN useradd --create-home --shell /bin/bash appuser
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY app.py .
RUN chown -R appuser:appuser /app
USER appuser
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

5. 03-three-tier/frontend/Dockerfile

One-line explanation: Multi-stage Dockerfile building React with Node 22 and serving via Nginx on port 80.

```
# Stage 1: Build static assets using Node.js 22
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json .
RUN npm install
COPY index.html vite.config.js ./
COPY src/ ./src/
RUN npm run build

# Stage 2: Serve static assets using Nginx web server
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

6. 03-three-tier/compose.yaml

One-line explanation: Declarative multi-container configuration orchestrating PostgreSQL, Flask backend, and React frontend.

```
services:
  database:
    image: postgres:17
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: apppass123
    volumes:
      - postgres-data:/var/lib/postgresql/data

  backend:
    build: ./backend
    environment:
      DB_HOST: database
      DB_PORT: 5432
      DB_NAME: appdb
      DB_USER: appuser
      DB_PASSWORD: apppass123
    ports:
      - "5000:5000"
    depends_on:
      - database

  frontend:
    build: ./frontend
    ports:
      - "3000:80"
    depends_on:
      - backend
```

7. 04-three-tier-nginx/compose.yaml

One-line explanation: Production Compose stack with an external Nginx proxy, frontend network, and an isolated internal database network.

```
services:
  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - frontend
      - backend
    networks:
      - frontend-network

  frontend:
    build: ./frontend
    depends_on:
      - backend
    networks:
      - frontend-network

  backend:
    build: ./backend
    environment:
      DB_HOST: database
      DB_PORT: 5432
      DB_NAME: appdb
      DB_USER: appuser
      DB_PASSWORD: apppass123
    depends_on:
      - database
    networks:
      - frontend-network
      - database-network

  database:
    image: postgres:17
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: apppass123
    volumes:
      - postgres-data:/var/lib/postgresql/data
    networks:
      - database-network

networks:
  frontend-network:
  database-network:
    internal: true

volumes:
  postgres-data:
```

8. 04-three-tier-nginx/nginx/default.conf

One-line explanation: Nginx reverse proxy routing web requests to frontend container and API requests to Flask container.

```
server {
    listen 80;
    server_name localhost;

    # Proxy root path traffic to the React frontend container
    location / {
        proxy_pass http://frontend:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Proxy API requests directly to the Flask backend container
    location /api/ {
        proxy_pass http://backend:5000/api/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

9. 04-three-tier-nginx/frontend/Dockerfile

One-line explanation: Frontend multi-stage Dockerfile serving compiled static assets only, leaving API routing to the main proxy.

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json .
RUN npm install
COPY index.html vite.config.js ./
COPY src/ ./src/
RUN npm run build

FROM nginx:alpine
# Static-only Nginx configuration (no /api/ proxy block needed)
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```